Need for Speed: High Stakes en Dreamcast

Estrategia de port y arquitectura To-Be

Proyecto Game_Development · 16 de octubre de 2026 · Base: github.com/bsimovic93/nfshs-rs

1. Resumen y decisiones

El objetivo es llevar Need for Speed: High Stakes (PS1, NTSC-U, SLUS-00826) a Dreamcast a partir del repositorio nfshs-rs, que ejecuta el código original del juego recompilado a Rust con un renderer nuevo. El usuario siempre aporta su propio disco original; el proyecto no distribuye datos del juego.

- **Enfoque:** recompilación estática de NFS4.EXE y FRONT.BIN, regenerada en **C** (no en Rust) y compilada con KallistiOS (sh-elf-gcc).
- **Render:** el PowerVR dibuja el 3D de forma nativa a partir de assets, como hace el renderer wgpu del repo. Es el segundo hito, después de que el juego compile y corra en Dreamcast.
- **Vídeo:** dos modos de salida, 480p a 31 kHz (VGA) y 480i a 15 kHz, con render 640×480 en ambos.
- **FPS:** los del juego original en PS1, sin subirlos. El hito 0 mide la cadencia real.
- **Audio:** se convierte antes de compilar y entra en el hito 3, junto con la optimización.
- **Hito 4, Enhancements:** escalado de texturas. **Hito 5:** motor NFSHS preparado, con el disco del usuario, y un pack de texturas nuevo distribuido por separado.
- **Legal:** el repo base no incluye archivo LICENSE; conviene pedir permiso al autor antes de copiar código (ps1rt, spu, formats). El recompilador no contiene datos del juego.

2. El repositorio base

nfshs-rs recompila estáticamente el ejecutable de PS1 a Rust y lo corre sobre un runtime mínimo. La lógica original (menús, física, IA, tráfico, policía, cámara y HUD) se ejecuta tal cual. El 3D y la presentación los dibuja un motor nuevo con wgpu, que lee el estado del juego en su RAM.

- **Tamaños aproximados:** tools/recomp.py unas 480 líneas; ps1rt unas 1.5k (memoria, GTE, BIOS, intérprete de respaldo); host unas 2.7k; spu.rs 676; renderer wgpu unas 1000 más 163 de shader.
- **El código recompilado** (nfs4rc) no está en el repo: se genera con el disco del usuario.
- **Render actual:** Render_RenderPlayerView (0x800B4024) está reemplazada por el host. El juego decide qué se ve y el renderer dibuja desde los assets.

NFS4.EXE no es un ejecutable de Windows

Es un **PS-X EXE**, el formato nativo de PS1: empieza con el texto PS-X EXE (un PE de Windows empieza con MZ), tiene una cabecera de 2 KB y después código MIPS R3000. Según la documentación del repo se carga en 0x80010000 y arranca en 0x800E4C38. La extensión .EXE viene de las herramientas de desarrollo de la época. El disco arranca así: SYSTEM.CNF indica SLUS_008.26 (cargador de unos 30 KB), que lanza NFS4.EXE. FRONT.BIN es un overlay con el front end que sigue residente durante las carreras. Por eso el recompilador necesita ambos archivos.

3. Recompilación estática

Consiste en traducir el programa original, antes de ejecutarlo, a código fuente (C o Rust) y compilarlo con un compilador moderno, reemplazando lo que el programa pedía al hardware por código del nuevo destino. **No es un emulador:** no interpreta la CPU, no hay BIOS real ni GPU, CD o DMA emulados, y las bibliotecas de Sony están reemplazadas por funciones del host. Sí conserva una RAM de 2 MB con el mapa de direcciones original, los registros MIPS, el GTE, un intérprete de respaldo para el código no cubierto y un modelo del SPU, porque el driver de sonido de EA lo programa por registros.

Frente a decompilar y volver a compilar

Aspecto	Recompilación estática	Decompilación
Esfuerzo	Semanas o meses; no hace falta entender cada función. En este repo ya está hecho.	Miles de horas, función por función.
Fidelidad	Comportamiento idéntico por construcción, incluidos bugs y matemáticas enteras.	Exige igualar bytes con el compilador original o aceptar diferencias.
Código resultante	No legible ni mantenible; cambiar la lógica del juego es difícil.	Código fuente real, legible y modificable.
Rendimiento en el destino	Arrastra el modelo de máquina original (RAM de 2 MB, punteros de 32 bits); el código crece.	Optimizado para el destino.
Cambio de plataforma	Cambiar el emisor del recompilador (aquí, a C).	Compilar para el destino.
Legal	Se distribuye la herramienta, no el código del juego.	El código reconstruido es obra derivada.

Estrategia híbrida: se recompila todo en los hitos 1 y 2 y, en el hito 3, se reemplazan las funciones calientes que se midan por versiones escritas a mano para SH-4, mediante el mecanismo de hooks que ya existe.

4. El renderer wgpu y qué implica para PVR

El renderer actual usa un solo atlas RGBA y un shader tipo PS1: textura × color de vértice × 2 (0x80 es neutro), color-key para transparencia y semitransparencia por texel, niebla lineal o la del GTE (DQA, DQB, H) y los faros nocturnos calculados por vértice como en el original. El orden de dibujo es cielo, pista estática (solo los rangos que elige el juego), objetos, sombras sustractivas, coches, reflejos sustractivos, destellos y HUD.

- La pista estática y los rangos de chunks visibles se mapean casi directo a buffers horneados.
- El color-key encaja con la lista punch-through del PVR y la semitransparencia con la lista translúcida.
- El PVR no tiene una operación de resta en sus modos de mezcla: sombras y reflejos sustractivos hay que aproximarlos, por ejemplo con quads oscuros mezclados por alfa.
- No hay shaders programables: el “× 2” y la niebla pasan al color de vértice, y los faros se calculan por vértice en SH-4. Con el color de vértice limitado a 1 se pierde brillo sobre el neutro; hay que medirlo en el hito 2.

5. Arquitectura actual (As-Is): nfshs-rs

Vista de aplicación del repo, solo software. Muestra cómo el juego recompilado corre sobre el runtime, cómo el host reemplaza las bibliotecas de PS1 y cómo el motor nuevo dibuja lo que el juego decide.

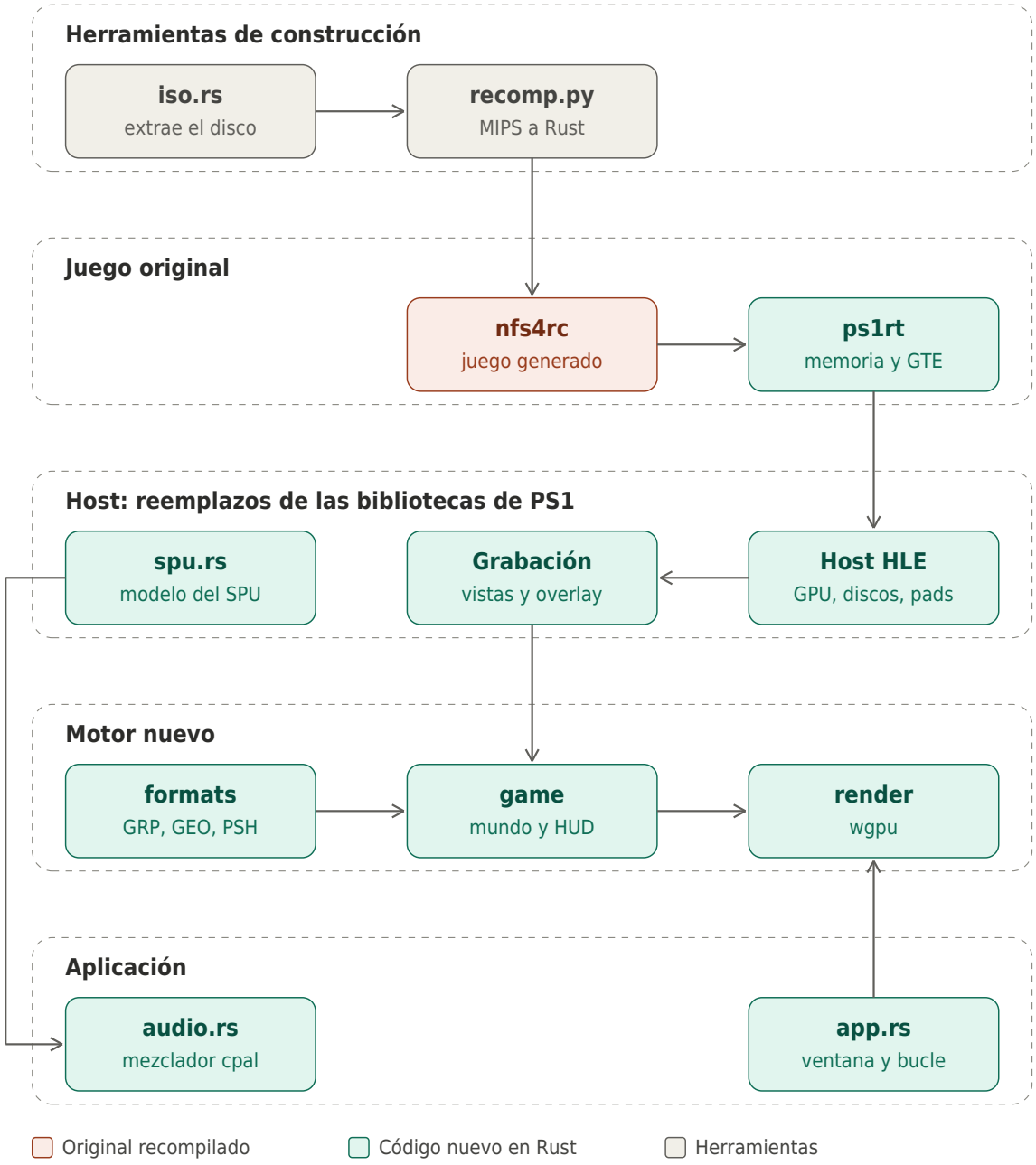


Figura 1. Vista de aplicación As-Is.

- **Herramientas:** iso.rs extrae el disco y recomp.py genera nfs4rc.
- **Juego original:** nfs4rc corre sobre ps1rt, que aporta memoria, GTE, BIOS y el intérprete de respaldo.
- **Host:** el Host HLE reemplaza GPU, discos, pads y timers; la Grabación registra cámara, chunks visibles, coches y 2D; spu.rs modela el SPU.
- **Motor nuevo:** game convierte lo grabado en mundo y HUD, formats decodifica GRP, GEO, PSH y VIV, y render dibuja con wgpu. El juego original corre en su propio hilo, un paso de 60 Hz por vez.

6. Arquitectura To-Be para Dreamcast (hitos 1 a 4)

Vista de aplicación del destino. Cada caja indica el hito en que aparece. Quedan fuera los dispositivos (consola, mando, pantalla) y lo que corresponde al hito 5.

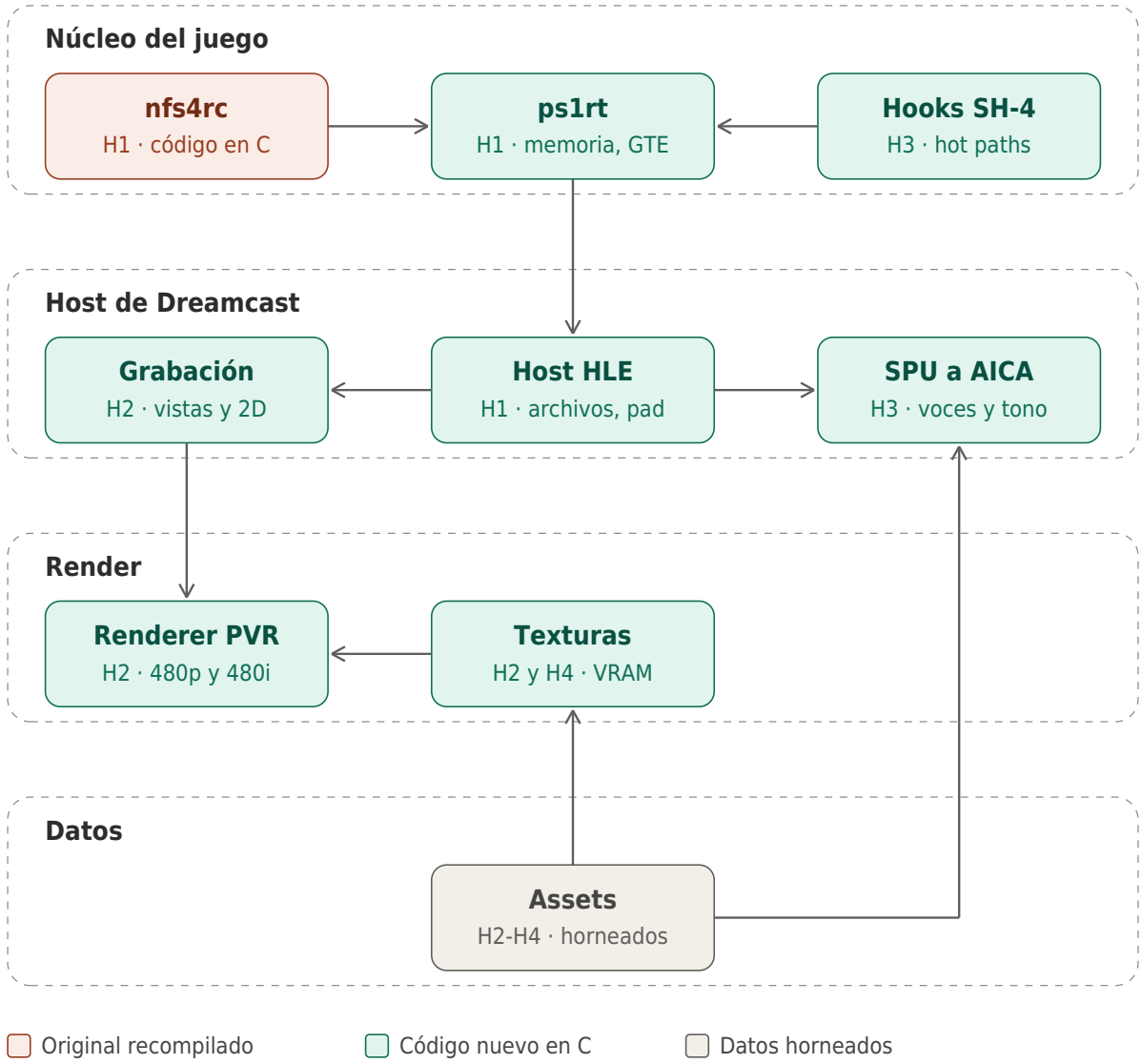


Figura 2. Vista de aplicación To-Be.

- **Núcleo del juego:** nfs4rc en C (H1) sobre ps1rt en C (H1). Los hooks SH-4 (H3) reemplazan funciones calientes y se registran en el despacho.
- **Host de Dreamcast:** el Host HLE (H1) atiende archivos, pad y timers; la Grabación (H2) pasa a C; SPU a AICA (H3) mapea voces del driver de sonido y el tono del motor a canales AICA.
- **Render:** el Renderer PVR (H2) dibuja en 480p y 480i con lo que recibe de la Grabación; el gestor de texturas (H2 y H4) administra VRAM y paletas.
- **Datos:** los assets horneados (H2 a H4) incluyen mallas, texturas en formato PVR, las versiones escaladas y las muestras de audio convertidas a AICA.

7. Pipeline de build

Del disco que aporta el usuario a un paquete para Dreamcast. Todo el procesamiento ocurre antes de ejecutar el juego.

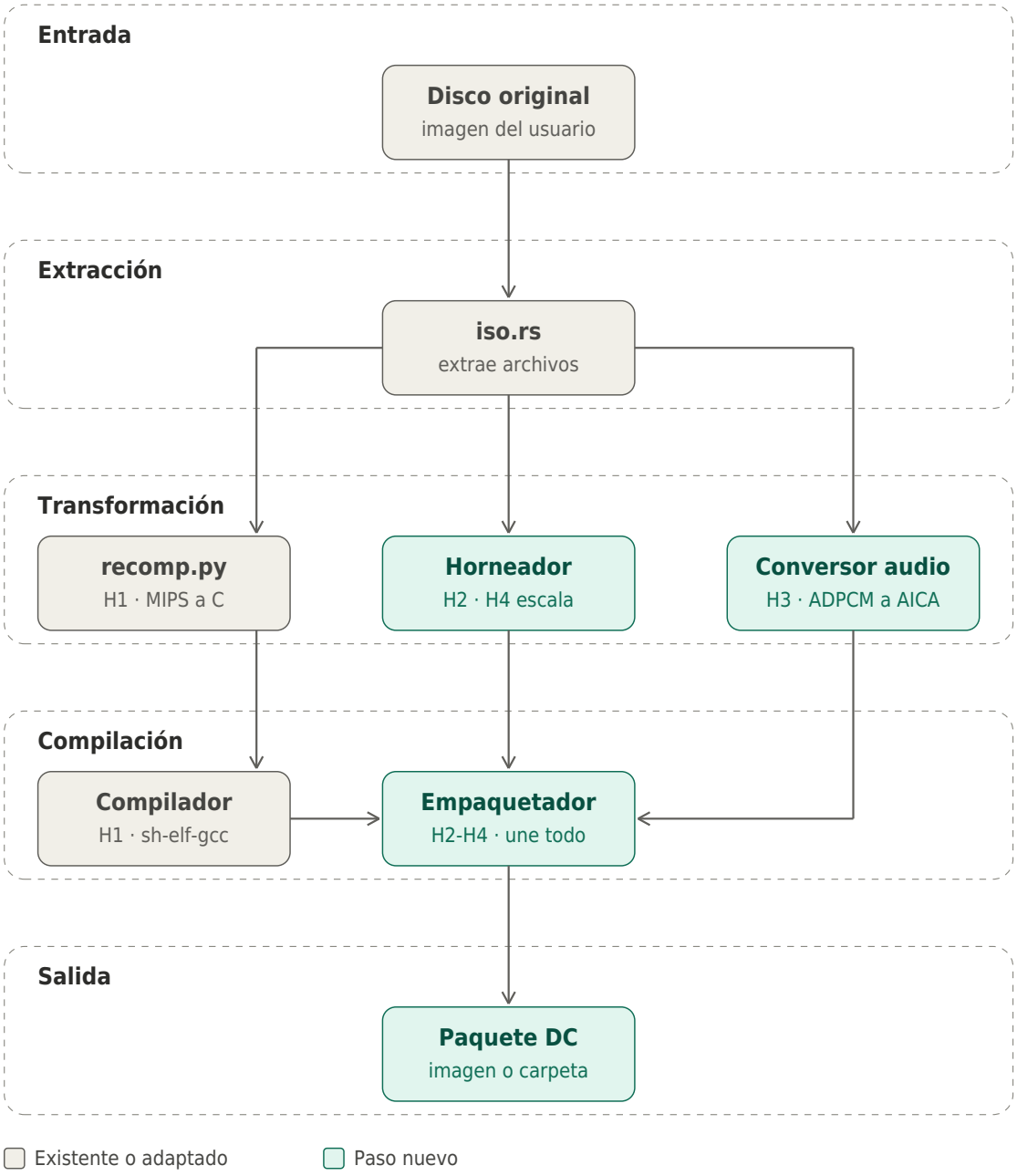


Figura 3. Pipeline de build (hasta el hito 4).

- **Extracción:** iso.rs saca los archivos de la imagen de disco.
- **Código:** recomp.py, con un backend nuevo, genera C a partir de NFS4.EXE y FRONT.BIN; el compilador lo une con el runtime y el host para producir el binario.
- **Assets:** el horneador convierte texturas y mallas a formato PVR (con el escalado del hito 4) y el conversor de audio pasa ADPCM de PS1 a un formato de AICA (hito 3).
- **Empaquetado:** une binario y datos en una imagen o carpeta lista para correr. En el hito 5 se suma el pack de texturas externo, que se aplica por encima.

8. Hitos

Hito	Objetivo	Criterio de salida
M0	Cifras de viabilidad en PC	Nº de funciones, tamaño del C generado, framerate original (VSync), uso de RAM PS1.
M1a	Backend C y ps1rt en C, en x86	Test diferencial contra la versión Rust: mismo input, mismo estado de RAM por frame.
M1b	Cross-compilar a SH-4 (KOS, -m4-single) y host mínimo	Compila y ejecuta en DC (Flycast y hardware): front end y lógica de carrera avanzan sin cuelgues, con contadores y estado del coche en pantalla; tamaño de ELF y RAM medidos.
M2a	Grabación en C, modos de vídeo 480p/31 kHz y 480i/15 kHz, 2D en PVR, espejo de VRAM y texturas	Front end navegable y legible en ambos modos.
M2b	Pista y cielo en PVR	Una pista recorrible; medir ms por frame y polígonos.
M2c	Coches, props y efectos	Carrera completa de un jugador.
M3	Optimización (SH-4, codegen, LOD y visibilidad) y audio	FPS original sostenido en carrera y menús; audio con muestras convertidas antes de compilar (SPU a AICA): música, motor y efectos.
M4	Enhancements: escalado de texturas	Texturas de pista, coches y HUD escaladas (horneadas en el PC) dentro del presupuesto de VRAM, con respaldo a las originales.
M5	Motor NFSHS preparado: VMU, rumble, FMV, 2P, pulido, build con el disco del usuario y packs de texturas externos	Motor jugable de punta a punta con el disco del usuario y un pack de texturas nuevo distribuido por separado.

9. Evolución: texturas y polígonos

Cambiar texturas es viable porque el juego recompilado decide la lógica pero no dibuja: el renderer arma mallas y atlas desde los archivos. La colisión sale de la “virtual road” (muestras cada unos 6 m) y no de la malla visible, así que se puede subir el detalle visual sin alterar la física.

- **Texturas:** se identifican por nombre en los PSH; una versión de 2× o 4× conserva los UV. Hay que respetar color-key, el bit STP, el modulado por color de vértice y las texturas animadas. El límite es la VRAM de 8 MB: se presupuesta por pista, con compresión VQ, paletas y mipmaps.
- **Coches:** pueden tener más detalle si conservan la estructura de partes, porque el juego decide cuáles se dibujan (luces, abolladuras, reflejos).
- **Pista y props:** se puede enriquecer la malla visual mientras coincida con los límites de la virtual road. Cambiar trazado o colisión exige modificar a la vez esas muestras, de las que dependen IA y cámara.
- **Costo:** más polígonos cuestan transformación en SH-4 y relleno; como se mantienen los fps originales, cada ampliación se mide.
- **Distribución:** los assets nuevos son arte propio y se publican como pack aparte, aplicado por nombre de textura sobre los datos del disco del usuario, sin incluir assets de EA.

10. Riesgos

- **CPU:** el cuello de botella pasa a ser la lógica recompilada y la transformación y recorte de la pista en SH-4. Mitigación: medir en M1b y M2b contra el fps original, FTRV y FIPR, caché de registros en el codegen.
- **Renderer nativo:** es el mayor trabajo nuevo (unas 3k líneas en Rust entre src/game y src/render, más la grabación del host). Mitigación: subetapas M2a a M2c y comparación contra el renderer de referencia en PC.
- **RAM:** 16 MB para 2 MB de RAM PS1, el espejo de VRAM, el código recompilado, buffers de audio y assets. Medir en M0 y M1b.
- **Funciones enormes en C:** GCC puede tardar o generar código pobre; dividir por bloques si ocurre.

- **VRAM del PVR:** riesgo de thrash en el caché de texturas, y de falta de espacio con texturas escaladas.
- **Audio:** el ADPCM de PS1 no es el de AICA, y el driver de EA programa el SPU por registros; hay que relacionar cada subida de muestras con su versión convertida (por banco y offset).
- **FMV y guardado:** las FMV (STR/MDEC) se decodifican por software o se recodifican; la partida va a un archivo en VMU.
- **Legal:** licencia del repo base y que nada de EA se distribuya.

11. Decisiones abiertas y próximo paso

- Cómo se presenta el ritmo original de fps en 480i y 480p, una vez medido en el hito 0.
- Escalado de texturas: factor (2× como base), método (algoritmo o IA, hecho en el PC), formato final y presupuesto de VRAM por pista.
- Alcance del hito 5: si 2P y FMV entran en la primera versión.
- Medio de pruebas en hardware (SD, ODE, imagen de disco) y contacto con el autor de nfshs-rs.
- **Próximo paso:** ejecutar el recompilador con el disco del usuario (NFS4.EXE y FRONT.BIN) para obtener las cifras del hito 0.

Fuentes

Repositorio nfshs-rs (github.com/bsimovic93/nfshs-rs): README, docs/FORMATS.md, tools/, src/render, src/ps1, src/game. Rust en Dreamcast: libanzen (github.com/spencerelliott/libanzen) y la imagen dc-kos-toolchain-rs (hub.docker.com/r/hldtux/dc-kos-toolchain-rs).